Computer software validation — RDM

Document ID	CSV-001
Revision	e635a5a
Date	2026-11-11
Status	proposed

Contents

1 Intended use 3
2 System Architecture 3
2.1 Context map 3
2.2 The four parts 4
2.3 System context 5
2.4 Bounded contexts (one design document each), by part 5
2.5 Dependency rule 6
2.6 Flow 7
2.7 Not yet in scope 7
3 Process risk 7
4 Risk policy 8
5 Risks 8
6 Specification 12
6.1 compliance 13
6.2 graph 15
6.3 publishing 21
6.4 release 25
6.5 risk 28
6.6 specification 34
6.7 test_evidence 38
7 Assurance activities 40
8 Record 40
9 The record system and Part 11 40
10 Change 41

Id: `CSV-001` · Status: `proposed` · References: `VVP-001, DC-001, SDS-SYS-001` · Kind: `risk`

Proposed, not yet approved. The validation of RDM as a tool: software used as part of a regulated team's quality management system, not a medical device, validated the computer software assurance way, risk based ([[13485:4.1.6-procedure]]). This is RDM's final document: its intended use, its architecture, its risk analysis (the risk policy and the risks are this document's frontmatter, which the release gate evaluates), its whole spec with each requirement's verification, the assurance, the record and change. It is a template: `rdm render` fills in the architecture and the spec from the record, with the `include` and `glob` filters, and the verification from `data/verification.yml`. A team that adopts RDM validates it for its own intended use; this record is the supplier's assurance it can rely on.

1 Intended use

RDM reads a team's spec, checklists, risks, documents and executed test results; reports the checklist clauses no document references (gap analysis); decides whether a release may go ahead (the release gate); and generates the verification report and the evidence bundle, which are quality records. It is used directly as part of the quality management system: it automates a quality process (the release decision) and maintains quality records ([[CSA:intended-use]]). It does not produce the product itself, so ISO 13485 7.5.6 applies to it only where a team uses its gates inside production.

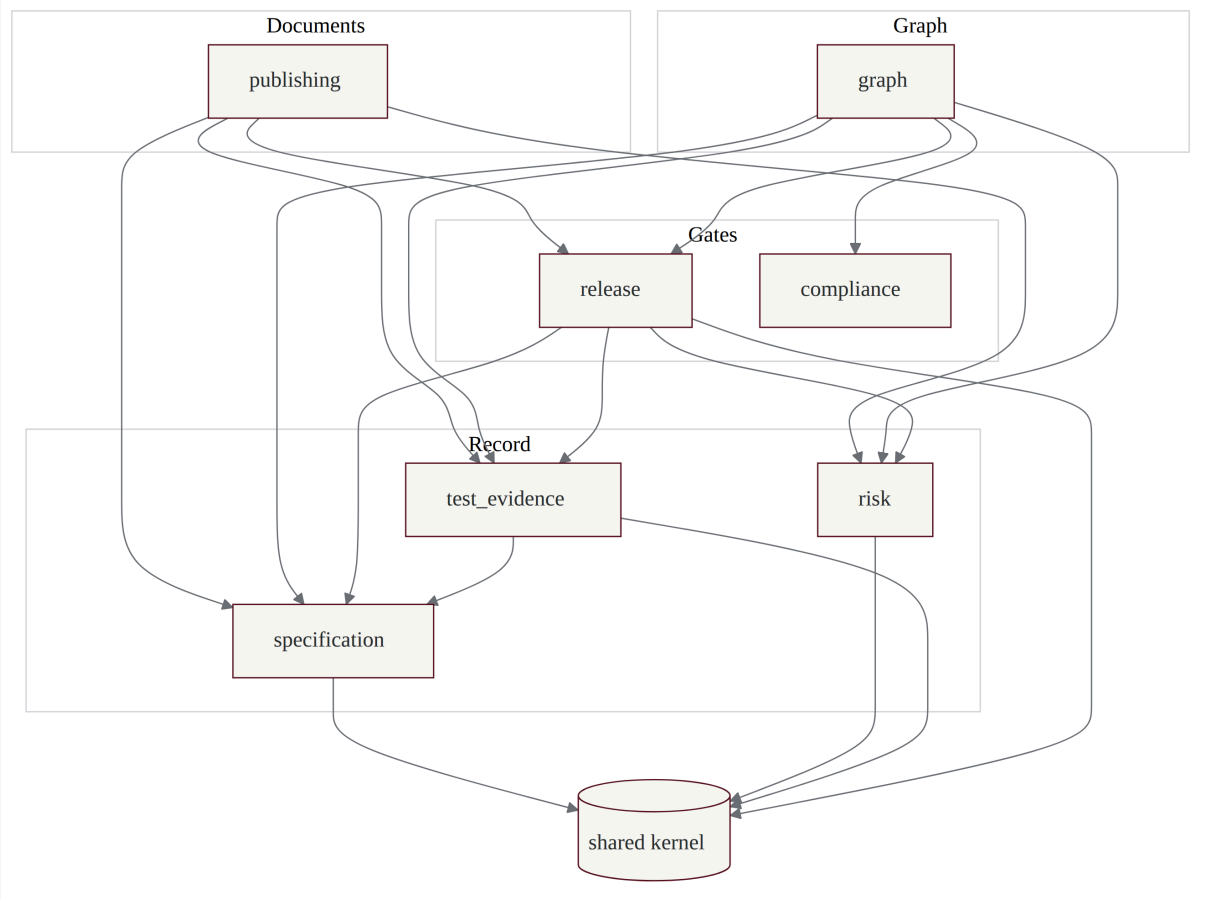
2 System Architecture

RDM audits the record of regulated software. It reads a spec as Gherkin (each scenario tagged with an id a requirement), takes the evidence of each requirement from Allure test runs, checks documents against the checklists of the standards that apply, decides whether a release may go ahead, renders regulatory documents, and builds it all into a read-only graph that people and agents query. It never authors the spec (ADR 0002).

This document holds **design only**: RDM's own requirements and user needs are its spec, written the Löyly way in `.pi/` (a slice of `.pi/plan/` per bounded context), read like any project's.

2.1 Context map

Each bounded context in its part; an arrow is a dependency in the code (the context imports the one it points to), downwards only, as the dependency rule requires (below).



2.2 The four parts

Part	Responsibility	Writes to the record?
Record	read the record: the spec (Gherkin), the risk register, validation records, checklists, test results, git	scaffolding only (`init`, `adopt`); every other change is a reviewed commit
Gates	pass/block decisions on the record	no
Graph	the knowledge graph: the record as RDF, queried and validated; agents read it over MCP (`rdm graph mcp`)	no — rebuilt from the record, never edited
Documents	regulatory documents rendered from the record	no

Only people and agents change the record, and only through a reviewed pull request; the merge is the approval. Nothing RDM derives (graph, matrix, documents, evidence bundle) is edited by hand or fed back into the record.

2.3 System context

Who uses RDM: a regulatory author renders documents and checks them against checklists; a team runs the gates in CI on every change; an agent reads the record through the graph's MCP server; a reviewer reads the verification report and the evidence bundle. RDM runs as the `rdm` command, as a pytest plugin inside a project's acceptance run, and as reusable CI (a GitHub workflow and actions). It depends on git, on Allure results written by the project's test runner, and, to render PDFs, on Pandoc and Typst.

2.4 Bounded contexts (one design document each), by part

A bounded context is drawn where the language changes, not where the workflow does (`CONTEXT.md`): the stages of a requirement's life (written, verified, released) are not contexts. The code of each context is in the frontmatter (`code`).

Part	Context	Design document	Its language	Package
Record	`specification` (core)	`design/specification.md`	spec, slice, scenario, requirement, user need, surface, screen	`rdm/specification/`: the Gherkin reader, the spec's configuration, the Löyly adapter, `init`, `adopt`, validation records
Record	`risk`	`design/risk.md`	hazard, harm, severity, probability, control, residual	`rdm/risk/`
Record	`test_evidence`	`design/test_evidence.md`	test run, executor, step, attachment, tag; Allure's words stop here	`rdm/evidence/`: Allure results; and `rdm/pytest_plugin.py`
Gates	`release`	`design/release.md`	verified, release gate, finding	`rdm/release/`: the release gate, the verification data; and the reusable CI

Part	Context	Design document	Its language	Package
Gates	`compliance`	`design/compliance.md`	standard, checklist, checklist clause, gap, coverage	`rdm/compliance/`
Graph	`graph`	`design/graph.md`	knowledge graph, projection, vocabulary, gate rule	`rdm/graph/`: the projection, the shapes, the SPARQL endpoint and the MCP server
Documents	`publishing`	`design/publishing.md`	template, data, rendered document, evidence bundle	`rdm/publishing/`: render, snippets, DMR index, the verification report, the evidence bundle; and `rdm/md_extensions/` and the PDF action

Not contexts: the **shared kernel** (`rdm/kernel/`: YAML and file helpers, EDN, ids, git, frontmatter, the reconcile helpers), below every context; and the **composition root** (`rdm/main.py`), which wires the command line to every context. Onboarding (`init`, `adopt`) is the specification's application layer. Three paths users name stay where they are: `rdm/main.py` (the `rdm` command), `rdm/pytest_plugin.py` (imported by a project's `conftest.py`) and `rdm/md_extensions/` (named in a project's `config.yml`).

2.5 Dependency rule

A context imports only contexts of a lower layer, and the shared kernel; nothing imports a context of the top layer. The layers and each context's code are in the frontmatter, and a test fails on any import that breaks the rule, and on any module no context's code holds (`tests/dependency_rule_test.py`).

```
  5  publishing · graph              read models: read every context, feed none back
  4  release                         decides: verified status, risk and validation →
release
  3  test_evidence                   translates Allure results into runs of the spec's
```

```
requirements
 2  specification                  the core: the spec read as Gherkin, needs,
requirements
 1  risk · compliance              leaves: their own languages, nothing but the
kernel
 0  shared kernel                  util · edn · ids · git · frontmatter · reconcile
```

The leaves sit below the core because the core uses them and they use nothing: the release gate applies the risk rules, and neither leaf reads a requirement. The evidence bundle renders the matrix and the verification report, so it is publishing's, and realises release's item-04-003.

2.6 Flow

The spec → its evidence (Allure runs) and its risks → the release gate decides, and gap analysis holds the documents to their checklists → the knowledge graph and the documents are derived from the same record. Agent skills (how to author requirements, test-first, risk analysis) are maintained outside RDM, in `scope-impact/agent-skills`. RDM ships no planning tooling: tasks and issues live in their own tools, outside the record (see `docs/about/plan-vs-record.md`).

2.7 Not yet in scope

In rough order: turning a regulation into a checklist mapped to its standard's clauses; pushing the graph to an external RDF store, with the vocabulary and gate shapes versioned; and several projects in one graph (clause identifiers are already project-independent). Each enters through a user need and requirements in the spec before any code.

3 Process risk

A failure of RDM to perform as intended could let a release go ahead with a requirement not verified, a risk not evaluated, or evidence that does not describe what was tested: RDM determines acceptability with limited additional human review where a team relies on its gate alone. That is a quality problem that could foreseeably compromise safety: **high process risk** ([[CSA:risk]], [[13485:4.1.6-proportionate]]). The ways it could fail are the risks below, evaluated against the risk policy below (severity from the harm, probability from the situation, the residual after the controls), each control a requirement of RDM's own spec. The analysis covers every factor the guidance names (its §V.A(2)), for failures that are reasonably foreseeable, not only likely:

Factor	Risks
Configuration and management	RISK-TOOL-008 (the spec read short)
Security	RISK-TOOL-004, RISK-TOOL-006, RISK-TOOL-007

Factor	Risks
Data integrity	RISK-TOOL-002, RISK-TOOL-005
Data storage	RISK-TOOL-009 (evidence not retrievable)
Data transfer, operation error	RISK-TOOL-010 (a decision on other results)

4 Risk policy

Severity / probability	Rare	Unlikely	Possible	Likely
Critical	Medium	High	High	Block
Serious	Low	Medium	High	High
Minor	Low	Low	Medium	Medium
Negligible	Low	Low	Low	Low

- **Low** — acceptable
- **Medium** — justify
- **High** — justify
- **Block** — unacceptable

5 Risks

Risk	Hazard	Before	After	Decision
RISK-TOOL-002	The release gate passes a requirement whose test failed or never ran.	High	Low	not evaluated
RISK-TOOL-004	An agent changes the controlled record through RDM without review.	High	Low	not evaluated

Risk	Hazard	Before	After	Decision
RISK-TOOL-005	A test that claims a requirement passes without checking everything its scenario states.	High	High	accepted
RISK-TOOL-006	The served graph accepts SPARQL updates from any web origin.	Medium	Low	not evaluated
RISK-TOOL-007	A read-only query can make RDM send network requests.	Medium	Low	not evaluated
RISK-TOOL-008	The spec is read short: requirements are left out and no one notices.	High	Low	not evaluated
RISK-TOOL-009	The evidence that a release was verified cannot be retrieved later.	Medium	Medium	not evaluated
RISK-TOOL-010	The gate decides on results that are not the evidence of the commit being released.	Medium	Low	not evaluated

RISK-TOOL-002 safety proposed High → Low

Hazard The release gate passes a requirement whose test failed or never ran.
Situation A test fails, errors or is skipped in CI, and the release gate still reports PASSED.
Harm A team releases software with a requirement that is not verified.
Rating Serious × Possible = High; after its controls Serious × Rare = Low: not evaluated

Controls

- `item-04-001` untested
- `item-04-002` untested

RISK-TOOL-004 security Tampering proposed High → Low

Hazard An agent changes the controlled record through RDM without review.
Situation An agent connected to rdm graph mcp issues a SPARQL Update or looks for a write tool.
Harm The record diverges from what reviewers approved, and evidence no one reviewed is presented as controlled.
Rating Serious × Possible = High; after its controls Serious × Rare = Low: not evaluated

Controls

- `item-06-007` untested

RISK-TOOL-005 safety proposed High → High

Hazard A test that claims a requirement passes without checking everything its scenario states.
Situation A test checks two of the three things a requirement requires, and the pull-request reviewer does not notice.
Harm The requirement is reported verified while one thing it requires is unchecked, and the gap ships.
Rating Serious × Possible = High; after its controls Serious × Possible = High: accepted

Controls none

> **Accepted** by RDM maintainers, by approving the pull request that records this register: Whether a test proves its requirement is a reviewer's judgement, made in the pull-request review; no gate can make it. Monitored by reviewers reading each changed test against its scenario's text.

RISK-TOOL-006 security Tampering proposed Medium → Low

Hazard The served graph accepts SPARQL updates from any web origin.
Situation A person browses the graph in Graph Explorer while a web page they have open posts an update to the local endpoint.
Harm The graph the person reviews is cleared or shows forged verification, and they act on evidence the record does not hold.
Rating Serious × Unlikely = Medium; after its controls Serious × Rare = Low: not evaluated

Controls

- `item-06-002` untested

RISK-TOOL-007 security Information disclosure proposed Medium → Low

Hazard A read-only query can make RDM send network requests.
Situation A prompt-injected agent, or a web page posting to the served endpoint, sends a query with a SERVICE clause naming an internal or external address.
Harm The server probes or reaches services from the user's machine, outside the review the record is under.
Rating Serious × Unlikely = Medium; after its controls Serious × Rare = Low: not evaluated

Controls

- `item-06-007` untested
- `item-06-002` untested

RISK-TOOL-008 safety proposed High → Low

Hazard The spec is read short: requirements are left out and no one notices.
Situation A context's spec is missing from the repository (ignored by git, or a plan link to a folder never committed), and the gate counts only what it found.
Harm A release passes with requirements that were never checked, reported as complete.
Rating Serious × Possible = High; after its controls Serious × Rare = Low: not evaluated

Controls

- `item-04-001` untested
- `item-01-001` untested

RISK-TOOL-009 safety proposed Medium → Medium

Hazard The evidence that a release was verified cannot be retrieved later.
Situation The CI artifact holding the evidence bundle expires, or the repository is lost.
Harm An auditor cannot confirm what was verified for a release, and the team cannot show it.
Rating Serious × Unlikely = Medium; after its controls Serious × Unlikely = Medium: not evaluated

Controls

- `item-04-003` untested

> **Accepted** by RDM maintainers, by approving the pull request that records this register: RDM writes the complete, checksummed bundle; keeping it for the retention period is document control's job (DC-001, 11.10(c)), not the tool's. Monitored through DC-001's retention commitment.

RISK-TOOL-010 safety proposed Medium → Low

Hazard The gate decides on results that are not the evidence of the commit being released.
Situation Results are altered or swapped on their way from the test job to the gate, or an earlier run's results are used.
Harm A release is approved on evidence of something else.
Rating Serious × Unlikely = Medium; after its controls Serious × Rare = Low: not evaluated

Controls

- `item-06-016` untested
- `item-07-009` untested
- `item-04-003` untested

6 Specification

RDM's spec, written the Löyly way in `.pi/spec/`: each accepted scenario, a concrete Given/When/Then example stamped `@item-NN-SSS`, is a requirement, verified by the acceptance test that claims it; each event-model screen, linked to its Allium surface, is a user need. The table is each requirement's verification from the executed results; the feature files follow, laid out by the `gherkin` render filter.

The spec was written after RDM's code, to describe it, so for this revision only the Gherkin is held to the code, by the tests that claim it; the Allium surfaces and event models are held to it by review. From the next change on, RDM is built the Löyly way, spec first and then regenerated, so the code cannot drift from any part of the spec.

The verification data is not part of this copy: run the acceptance tests and `rdm story verify` (`data/verification.yml`), then render it again.

6.1 compliance

Feature: Compliance

The accepted scenarios of this context: gap analysis of documents against the checklists of the applicable standards, and the checklists RDM ships. Each is stamped with an item id that is never renumbered or reused, and verified by the acceptance tests whose Allure results carry that id as a tag label.

RULE **A document set has a gap for every checklist key no document references**

`item-05-001` **Gap analysis names each missing reference and exits non-zero when one is missing or nothing was checked**

Given a checklist “cl.txt” with the clauses X-1 and X-2
When the regulatory author runs rdm gap cl.txt over “partial.md”, which references [[X-1]] only
Then it exits 3, listing X-2 and not X-1, under the heading “# 1 gap:”
And over “full.md”, which references [[X-1]] and [[X-2]], it exits 0
And a stray “[[“ does not turn the next paragraphs’ mentions into references: X-2 is still missing
And a bare mention “We do not address X-2” is no reference to X-2: it exits 3
And [[X-12]] does not reference the key X-1: it exits 3
And [[FDA-SW:sdmp: This document is a pointer document.]] references the key FDA-SW:sdmp: it exits 0
And [[FDA-SW:sdmp]] does not reference the key FDA-SW: it exits 3
And a byte-order mark is not part of the first key: a checklist beginning with a byte-order mark before “X-1 first” is covered by full.md
And “include cl.txt” with spaces around the name is resolved
When nothing could be checked: a checklist with no clauses, a checklist that does not exist, a bare “include”, an include of a file that does not exist, a document that does not exist, or a directory given as a document
Then it exits 2, never 0
When a document covering every clause still holds a “TODO: write this section” placeholder
Then it exits 0 and warns, naming the document and its placeholder
When its report of 20000 gaps is piped into a reader that stops after one line
Then it ends quietly, with no traceback and no BrokenPipe error

Covers: rule Gap (success) · rule Gap (refusal 1) · rule Gap (refusal 2) · rule Gap (refusal 3)

RULE **RDM ships composable checklists for the applicable standards**

`item-05-002` **The built-in checklists are listed by name, hold unique keys and resolve their includes**

When the regulatory author lists the built-in checklists

Then 62304_2015_class_b, 14971_2019, FDA-SW_2021_enhanced and csv_tool_validation (computer software validation of a tool) are listed by name
And IEC 62304, ISO 14971, FDA-SW, FDA-CYBER and FDA-HFE each have a built-in checklist with clauses
And no built-in checklist holds a key twice
And 62304_2015_class_b, which includes 62304_2015_class_a by name, requires every clause of class A and more
Given a checklist “main.txt” with “include base.txt” and the clause M-1, and “base.txt” with the clause B-1
When a document referencing only [[M-1]] is checked against main.txt
Then it exits 3: B-1, defined only in the included file, is still required
And a document referencing [[M-1]] and [[B-1]] exits 0
Given “a.txt” includes “sub/b.txt”, which includes “../a.txt” and “./../a.txt”
When the coverage of a document referencing [[A-1]] is reported against a.txt
Then a.txt is read once: its row is “| A.TXT | 2 | 1 | 1 | 50% |” and it exits 3

Covers: guarantee BuiltInChecklistsCompose

RULE **Coverage of a document set is tabulated per checklist**

`item-05-003` **The coverage report gives each checklist’s total, missing, covered and percent**

Given a checklist “iso_checklist.txt” with the clauses ISO-1, ISO-2 and ISO-3
When the regulatory author reports coverage of “process.md”, which references [[ISO-1]] and [[ISO-3]]
Then the row “| ISO | 3 | 1 | 2 | 66% |” is printed and it exits 3
And in verbose mode the missing ISO-2 is named
When the coverage of a document referencing all three clauses is reported
Then it exits 0, and in verbose mode prints no empty “Missing Items” heading
When the checklist has no clauses or does not exist
Then it exits 2: nothing could be checked

Covers: rule Coverage (success) · rule Coverage (refusal 1)

RULE **RDM keeps its own record under the 21 CFR Part 11 checklist it ships**

`item-05-004` **RDM’s own document-control statement passes the built-in Part 11 checklist, and fails without a control**

When the regulatory author lists the built-in checklists
Then part11_document_control is listed by name
When dhf/documents/document_control.md is checked against part11_document_control
Then it exits 0: the statement references every clause
When a copy of the statement without its reference [[P11:11.10e]] is checked
Then it exits 3: dropping one control fails the audit

Covers: guarantee Part11ChecklistShipped

6.2 graph

Feature: Graph

The accepted scenarios of this context: the record projected into an RDF graph that people query and explore and agents read, never edit. Each is stamped with an item id that is never renumbered or reused, and verified by the acceptance tests whose Allure results carry that id as a tag label.

RULE **The record is projected into one named graph per source, deterministically**

`item-06-001` **A committed record becomes a dataset of named graphs, byte-identical while the record is unchanged**

- **Given** a committed repository whose spec holds the needs Alarm and Logging as Features and the requirements item-01-001 and item-01-002 under them
- **And** controlled documents with id, title and revision, and Allure results
- **When** the author builds the graph
- **Then** the record graph holds each user need with its Feature's title as id and its description as text
- **And** each requirement with its text, the needs it addresses, its context and its source line, and each bounded context
- **And** read through an adapter (Löyly's layout, Allium), the graph holds what a need faces, provides and guarantees, the rules a requirement covers, the Allium rules, and a context with no requirement
- **And** each controlled document with its id, title and revision
- **And** an executions graph only when results are given
- **And** a git graph with the design document's latest commit, its time and its author
- **And** an rdfs:label on every typed node
- **And** an ontology graph of RDM's vocabulary, reusing OSLC RM
- **And** the dataset is written as sorted N-Quads, byte-identical for an unchanged record
- **And** a literal holding a Unicode line separator stays one statement

Covers: guarantee RecordProjectedPerSource

RULE **The projection is stored, queried and served read-only**

`item-06-002` **The store is replaced on each build, queried with SPARQL and served read-only**

- **Given** a graph built into a store from a record with item-01-001 and item-01-002
- **When** the author queries the store with the standard prefixes undeclared and no GRAPH clause
- **Then** the query answers over the union of the named graphs
- **When** item-01-002 is removed and the graph is built again
- **Then** the store holds no item-01-002: it was replaced, not merged
- **And** with no store given, the query answers from an in-memory projection, ASK and CONSTRUCT included
- **And** a SPARQL Update or a SERVICE call is refused: nothing changes, nothing reaches the network

And FROM, which the union default graph would ignore, is refused, and so is SERVICE with a variable endpoint, while a prefix named service is no call
When the store is served over HTTP
Then the endpoint's default graph is the union, the standard prefixes are predeclared and CORS is open
And an update from another origin is refused and the data stays
And SERVICE is refused, even after a comment ended by a carriage return
And the endpoint answers from the last build
And a store that does not exist is refused, not created

Covers: guarantee StoreQueriedAndServedReadOnly

RULE **Checklists join the graph as data**

`item-06-003` **A text checklist, a built-in one and an RDF one become SKOS collections of clauses**

Given a text checklist that includes a built-in checklist by name, and a checklist written in RDF
When the author builds the graph with those checklists
Then each checklist is a SKOS collection of its own clauses that links the checklists it includes
And the built-in checklist is resolved by its name
And each clause has its key, description, standard, edition and nearest listed parent clause
And the RDF checklist is loaded as it is, its nodes labelled like any other

Covers: guarantee ChecklistsAsSkos

`item-06-009` **A document's reference tags link it to the clauses rdm gap would match**

Given documents whose [[...]] tags reference some clauses of a checklist and not others
When the author builds the graph with that checklist
Then each document links to the clauses its tags reference, matched as rdm gap matches keys
And the clauses no document references are exactly those rdm gap reports missing

Covers: guarantee DocumentsLinkTheirClauses

RULE **The gate rules hold over the graph as SHACL shapes**

`item-06-004` **The shapes block exactly what the release gate blocks, each finding on its node**

Given records with the requirements item-01-001 and item-01-002, each with these needs and runs:

case	needs	runs	blocked
all pass	Need1, Need2	item-01-001 passed; item-01-002 passed	nothing
failed	Need1, Need2	item-01-001 passed and failed; item-01-002 passed	item-01-001
broken	Need1, Need2	item-01-001 passed and broken; item-01-002 passed	item-01-001
untested	Need1, Need2	item-01-001 passed	item-01-002
skipped only	Need1, Need2	item-01-001 passed; item-01-002 skipped	item-01-002
unaddressed need	Need1, Need2, Need3	item-01-001 passed; item-01-002 passed	Need3

When the shapes are validated over each record's graph and the release gate is run on it
Then for each case both block exactly what the table names
When the needs are Need1, Need2 and Need3 and item-01-001's only run failed
Then Need3 is a violation "user need is addressed by no requirement"
And item-01-001 is a violation "requirement has a failed or broken test run"
And item-01-002 is a violation "requirement is not verified by any passing test run"
And a checklist clause no document references, and a clause without a key or a standard, are violations
And a requirement addressing no user need, and a run claiming an id that is no requirement, are warnings
And what the release gate blocks about the whole record, the shapes block too
And the executions graph reads results as the gates do: a byte-order mark is read, a failed step fails its run

Covers: guarantee GateRulesAsShapes

`item-06-010` **rdm graph validate reports each result and exits on a violation**

Given a record whose graph has violations, and one without
When the author runs rdm graph validate on each

Then each result is a line with its severity, focus node and message
And the command exits 1 on the record with violations and 0 on the other
And a user-supplied shapes file adds its rules as data
When a shapes or checklist file is not RDF
Then the command reports an error and exits 2, never with a traceback

Covers: rule GraphValidate (success) · rule GraphValidate (refusal 1) · rule GraphValidate (refusal 2)

RULE **The graph can be opened in AWS Graph Explorer as a file**

`item-06-005` **The whole record is written as a Graph Explorer file, a class left out on request**

Given a record with needs, requirements, documents, risks and test runs
When the author writes the graph as a Graph Explorer file
Then the file has Graph Explorer's envelope and connection
And every node of the record, of every kind, and nothing from the vocabulary
And every link between two record nodes, in Graph Explorer's edge-id form, and no rdf:type statement
When a class is left out
Then its nodes and their links are dropped, with the nodes that hang only from them, and only those
And the command writes the file from a fresh projection or from a built store
When a class to leave out is one the graph does not have
Then it is refused, naming the classes the graph has

Covers: rule ExplorerFile (success) · rule ExplorerFile (refusal 1)

RULE **An agent reads the record through four read-only tools**

`item-06-006` **The agent server answers schema, query, trace and validate from the current record**

Given a record with the needs Alarm and Logging, the requirements item-01-001 and item-01-002, and a passing run of item-01-001
When an agent connects to rdm graph mcp
Then it is offered exactly four tools: schema, query, trace and validate
And schema gives the vocabulary and the predeclared prefixes
And query answers SPARQL with those prefixes predeclared
And trace of item-01-001 gives its scenario, context, source, needs, coverage and runs
And trace of a need gives its text, context and requirements, and what an adapter adds (faces, provides, guarantees) is empty
And validate gives the gate shapes' results: item-01-002 has no passing run
When the record is edited during the session
Then the next call answers from the edited record

Covers: rule ServeAgent (success)

`item-06-007` **The agent server cannot change the record or reach the network**

Given the agent server over a record

Then its four tools are all declared read-only and non-destructive
When the agent sends every form of SPARQL Update
Then each is refused and nothing in the record changed
When the agent sends SERVICE in any case, with an IRI or a variable endpoint
Then it is refused, while the word inside a literal is just text
When the agent traces input that is not id-shaped
Then it is refused as “not an id”
And trace answers an id the record declares whatever its shape, such as the need Logging, and nothing built from it
And SELECT, ASK, CONSTRUCT and DESCRIBE all work
And rows beyond the cap are cut and the cut is reported; under the cap nothing is flagged

Covers: guarantee AgentCannotChangeTheRecord

RULE **Risks are in the graph, with shapes that block what the release gate blocks**

`item-06-008` **Each risk’s facts are in the risks graph, and the shapes block the risks the gate blocks**

Given a risk register with safety and security risks, their scores, controls and acceptances, and a risk policy
When the graph is built
Then the risks graph holds each risk’s facts: category, STRIDE category, linked risks, hazard, situation, harm, scores, levels, residual decision, status, controls and acceptance
And the residual decision is “not evaluated” until every control has a passing test
And trace gives a risk with each controlling requirement, and a requirement with the risks it controls
And the shapes block exactly the risks the release gate blocks, case by case
When the risk policy is malformed
Then the gate blocks, and the shapes block every risk with its message
When a risk has no id
Then the gate blocks it, and so do the shapes, on a stand-in node
When a finding about the register has no risk to carry it
Then the gate blocks, and so do the shapes, on a stand-in node for the register

Covers: guarantee RisksProjectedWithShapes

`item-06-012` **A risk links to the document of the policy it was evaluated against**

Given a risk register and a document declaring the risk policy
When the graph is built
Then each risk links to the policy’s document
And without a policy, a risk links to none

Covers: guarantee RiskLinkedToItsPolicy

RULE **Each controlled document carries who landed its latest change**

item-06-011 **A document's latest change is recorded with the commit that landed it on the default branch**

- **Given** a change to SDS-1 on a feature branch, which goes on after the change while main moves on
- **When** someone else merges the branch into main, and later work lands on main
- **Then** SDS-1's latest change is authored on the branch and landed by the merge, with the merge's author
- **And** the landing is still the merge, not the newest commit
- **When** a change is on a branch that has not landed
- **Then** the document has no landing facts and a warning names it
- **When** the latest change is a direct commit on main
- **Then** it landed in itself, by its own author, and nothing warns
- **And** every controlled document has its commit and landing, the V&V plan as well as the design documents
- **And** in a repository with no remote whose one branch is neither main nor master, that branch is the default

Covers: guarantee LandingCommitRecorded

RULE **A run carries what it recorded, and nothing the record already holds**

item-06-013 **A run's steps and attachments are in the graph and in trace**

- **Given** an Allure result with nested steps and attachments on the run and on a step
- **When** the graph is built
- **Then** the run carries its steps in order and nested, each with its name and status
- **And** its attachments, on the run and on the step, each with its name, media type and file
- **And** trace lists them with the run, nested and ordered

Covers: guarantee StepsAndAttachmentsCarried

item-06-014 **A result is projected in full, but not what claims nothing about design controls**

- **Given** a failed Allure result with a uuid, a full name, start and stop times, a status message and trace, parameters, labels and links, and a container with fixtures
- **When** the graph is built
- **Then** the run carries its uuid, name and times, its status message and trace, and its parameters as name and value
- **And** its labels are not nodes, its links, a test case per history id and the container fixtures are not projected
- **And** its item tag and output labels still link the run to its requirement and its code

Covers: guarantee OnlyDesignControlFactsProjected

RULE **Bounded contexts and documents link to each other from the record**

item-06-015 **Contexts link to the documents that declare them, and documents to the ones they reference**

Given design documents whose frontmatter declares contexts with their part and references other documents
When the graph is built
Then each bounded context links to the document whose contexts frontmatter declares it, with its part
And each controlled document links to the controlled documents its references frontmatter names
And the traceability matrix template, an output, is not projected
And once any document declares contexts, a shape warns on a context no document declares
And validation fails on a reference to a document the record does not hold
And a context may be declared by id alone, and a single reference may be a string

Covers: guarantee ContextsLinkedToTheirDocuments

RULE **Evidence is tied to the version of the product it tested**

`item-06-016` **Each run links to the commit it tested, and a run of another commit than the product's is stale**

Given a record at commit HEAD, and runs labelled with HEAD (one clean, one dirty), with an older commit, and with none
When the graph is built and validated
Then each run links to the commit its commit label names
And the record carries the commit it was built at and the product's commit, its own
And the run of uncommitted changes is marked and the clean one is not
And a shape warns on the run tied to no commit
And a shape warns on the run of the older commit: another commit than the product's
When the spec lives in a repository of its own
Then the product's commit is that repository's, the record's stays its own
And a run of the product's commit is current, and a run of the record's commit is stale

Covers: guarantee EvidenceTiedToTheProductCommit

6.3 publishing

Feature: Publishing

The accepted scenarios of this context: compiling the design history file from the record (templates and data to Markdown, its filters, its post-processing, snippets from source), RDM's own user manual, and the generated release documents (the device-master-record index and the verification report). Each is stamped with an item id that is never renumbered or reused, and verified by the acceptance tests whose Allure results carry that id as a tag label.

RULE **A document is a Markdown template rendered against the data files supplied, never against a value they lack**

`item-07-001` **rdm render fills a Markdown template from data files, each under its file name**

Given a template doc.md “# {{ device.name }}” then “Version {{ device.version }}, by {{ other.maker }}.”
And the data files empty.yml, device.yml with name “Acme Monitor” and version “1.2”, and other.yml with maker “Acme Inc”
When the regulatory author runs rdm render doc.md with empty.yml, device.yml and other.yml
Then the output is “# Acme Monitor”, then “Version 1.2, by Acme Inc.”
When other.yml is left out
Then rendering is refused with exit status 2, naming “other” and doc.md
When the template nope.md does not exist
Then rendering is refused with exit status 2, naming nope.md

Covers: rule Render (success) · rule Render (refusal 1) · rule Render (refusal 2)

RULE **Templates have the filters a traceability table needs**

`item-07-002` **invert_dependencies, join_to, md_indent, glob, include and gherkin build traceability tables and final documents**

Given items with id A depending on B
When a template inverts them with invert_dependencies(‘id’, ‘deps’)
Then B is depended on by [‘A’]
Given a table with rows r1 (v 1) and r2 (v 2)
When a template joins [‘r2’, ‘r1’] to it with join_to
Then it reads the rows in that order: “2;1;”
When a template indents the snippet “# Title” with md_indent(header_shift=1)
Then it reads “## Title”
Given feature files spec/b/spec.feature (“Feature: B”) and spec/a/spec.feature, and a document arch.md with frontmatter id SDS-1 and the heading “# Architecture”
When a template includes every file ‘spec/*/spec.feature’ | glob gives, and ‘arch.md’ | include | md_indent(1)
Then it carries spec/a/spec.feature, then spec/b/spec.feature, each as written, and “## Architecture” without the frontmatter
Given a feature file with a Rule “Within 2 s”, a scenario tagged @smoke @allium-rule-success.Raise @REQ-12 whose steps hold “$140” and “<value>”, an outline @REQ-13 with Examples, and a comment
When a template lays it out with gherkin, given verification data in which REQ-12 is verified by 2 runs
Then it is Markdown marked for the templates: the Rule a rule-line, each scenario a scenario-card led by its id and, for REQ-12, “✓ verified · 2 runs”, its steps’ keywords marked, the outline’s Examples a table, what its tags cover in words (“rule Raise (success)”, “need Alarm”), “$” and “<” escaped, and the comment left out
When rdm render joins the id r9 to a table holding only r1
Then rendering exits with status 2, naming r9

Covers: guarantee TraceabilityFilters

RULE **Rendered Markdown is post-processed: numbered, given its own words, and stripped of auditor-only notes**

item-07-003 **Sections are numbered, a second render gets the first's words, and auditor-only notes are removed**

- **When** "## hello" is rendered with section numbering
- **Then** it reads "## 1.1 hello"
- **When** "Spec [[1234:9.8.7.6]]." is rendered with auditor-note exclusion
- **Then** it reads "Spec."
- **When** a template lists first_pass_output.words after the words apple and banana, rendered with the vocabulary extension
- **Then** its second render reads "[apple][banana]"
- **When** numbering and note exclusion run together
- **Then** code in fenced blocks, tildes and inline code is left exactly as written, its "# a comment" unnumbered and its "[[1, 2]]" kept
- **And** only ATX headings are numbered: "#hashtag" and a lone "#" stay, "# Top" and "## Sub" become "# 1 Top" and "## 1.1 Sub"
- **And** a note over two lines of a paragraph, or nested in another, is removed whole: "A [[auditor" "only]] B." reads "A B."
- **And** a code fence inside a blockquote or a list item is code too, its notes kept
- **And** a note still open at the paragraph's end, "Stray [[here", is kept as text

Covers: guarantee MarkdownPostProcessed

RULE **Code snippets are extracted from source files for inclusion in documents**

item-07-004 **The lines between RDOC and ENDRDOC are a snippet under its key**

- **When** the lines "RDOC greeting", "hello", "world", "ENDRDOC" are collected
- **Then** the snippet greeting is "hello" then "world"
- **And** lines with no markers give no snippets
- **And** a marker is a whole word: "PRDOC_LIMIT = 3" starts no snippet, and the snippet limit is "LIMIT = PRDOC_LIMIT"
- **When** a.py and b.py both hold a snippet keyed "key"
- **Then** collecting them is refused, naming both files
- **And** rdm collect exits with status 2, naming b.py

Covers: guarantee SnippetsFromSource

RULE **RDM's user manual is one controlled document, published per release, that discloses every risk**

item-07-005 **IFU-001 lists the manual's pages in the reading order of instructions for use, behind a cover**

- **Given** the controlled document IFU-001 and the pages it lists
- **Then** its chapters appear in this order: "RDM documentation", "What RDM is for", "How it works", "Limits and risks", "Install", "Quick start", "Guides", "Troubleshooting", "Reference", "Changelog"
- **And** every other page sits in the folder of the chapter page before it
- **And** the site's Docs tab shows exactly these pages, in this order

And the first page is the cover, naming the product, “{{ rdm.release }}”, and “IFU-001, revision” with the manual’s revision
When the docs are built for the release tag v9.1.0
Then “{{ rdm.release }} @{{ rdm.ref }}” reads “v9.1.0 @v9.1.0”
When they are built for the development version
Then it reads “development version” and the install ref “@main”
And no page pins a release by hand

Covers: guarantee ManualInReadingOrder

`item-07-006` **The manual is published once per release tag and once for the development version**

Given the docs workflow and the site’s configuration
Then a release tag “v*” and a push to main both publish
And a tag publishes under its own name and as latest, main as the version dev
And every published version is kept on one branch, never replaced, and selectable
And the newest release, latest, is the default

Covers: guarantee PublishedPerRelease

`item-07-007` **The manual’s “Limits and risks” chapter discloses every risk of the register by its id**

Given RDM’s risk register and the manual’s “Limits and risks” chapter
Then the chapter’s risk table names every risk of the register by its id, and no other
And each row says what the user must do about the residual risk

Covers: guarantee RisksDisclosedById

RULE **The device-master-record index and the verification report are generated from the record**

`item-07-008` **The DMR index has one entry per controlled document, from its frontmatter**

Given documents/sop.md with id SOP-1, revision 2, title “The SOP”
And documents/plan.md with id PLAN-1, revision 1, title “The Plan”
And documents/notes.md with no frontmatter
When the DMR index data is generated
Then it lists PLAN-1 “The Plan” documents/plan.md revision 1, then SOP-1 “The SOP” documents/sop.md revision 2
And notes.md is named as skipped
When the only documents have a blank id or an empty one
Then nothing is indexed, a.md is named, and generation exits with status 1
And the output is marked GENERATED, and generating it again gives the same file

Covers: guarantee DmrIndexFromFrontmatter

item-07-009 The verification report shows a reviewer who did not run the tests what was verified

Given a record with requirements, risks and Allure results of passing, failing and skipped runs with steps and attachments
When the verification report is written
Then its header names the repository, the record's commit, the commits tested, the executor and environment, the RDM version and one SHA-256 over the result files
And the evidence is release-grade only when every requirement passed at the product's commit with a clean worktree; otherwise each reason is named
And its anomalies list failed and skipped runs, requirements with no run, missing attachments and orphan tags
And its traceability lists each requirement in spec order with its user needs, the risks it is a control for and its tests
And each requirement is a baseline or a risk-based acceptance criterion, each risk shows its status and residual decision, and the register's state is summarised
And each run shows its test's file and function, result, date and duration, failure message and trace
And it leaves out the labels it already shows, runner internals and Allure's links
And each step is a verification step, nested, with its own result
And attachments show text inline up to a limit and images embedded, other files listed, the captured output and the copy of the requirement listed by SHA-256 only
And the PDF shows all of it in that order, and an appendix lists every result file with its SHA-256
And text from a test reaches the page as text: markup in it is shown, not run
And without the typst package a typst executable is used, and with neither the report says so
And the evidence bundle includes the report, rdm story evidence-report writes it, and its SHA-256 can be checked from the bundle's copy of the results
And a result it cannot lay out leaves the reason in the bundle, not a crash
And a symbolic link among the results is never read: its attachment is missing and its file unhashed
And every run counts, tagged with a declared requirement or not; an unreadable or linked result, and uncommitted changes to the record, are each a reason it is not release-grade
Given a record whose spec lives in a repository of its own, and a passing run of that repository's commit
When the verification report is written
Then the product's commit is the spec repository's, the record's stays its own, and the header names both
And the run is release-grade evidence

Covers: guarantee VerificationReportForAReviewer

6.4 release

Feature: Release

The accepted scenarios of this context: what holds a release back, how a requirement's status is traced from executed results, what a release retains, and RDM's gates for other repositories' CI. Each is stamped with an item id that is never renumbered or reused, and verified by the acceptance tests whose Allure results carry that id as a tag label.

RULE **A release is blocked until the spec is read whole and every requirement has a passing run**

`item-04-001` **The release gate passes only when every requirement has a passing run and none fails**

Given a spec whose feature file stamps item-01-001 and item-01-002, both addressing the need Release
When the release manager runs the release gate with no Allure results
Then release is blocked, item-01-001 and item-01-002 untested
When a passing run tagged item-01-001 is added
Then release is blocked, naming item-01-002 and not item-01-001
When a failing run tagged item-01-002 is added
Then release is blocked: item-01-002 failed
When item-01-002's run passes and a failing run tagged item-01-001 is added
Then release is blocked, naming item-01-001: one passing run does not outweigh a failing one
When the failing run of item-01-001 is removed
Then release passes
When an unstamped scenario "an unstamped behaviour" is added to the feature file
Then release is blocked, naming "an unstamped behaviour"
When a second scenario is stamped item-01-001 instead
Then release is blocked: item-01-001 is stamped twice
When a feature file "broken.feature" that Gherkin cannot parse is added instead
Then release is blocked, naming broken.feature
And a slice of a Löyly plan whose spec.feature is missing (a link to a folder git never received) blocks release, naming the slice
When the feature file holds no stamped scenario
Then release is blocked: there is nothing to verify
And a repository with no spec at all is blocked: no spec was found

Covers: rule ReleaseGate (success) · rule ReleaseGate (refusal 1) · rule ReleaseGate (refusal 2) · rule ReleaseGate (refusal 3) · rule ReleaseGate (refusal 4) · rule ReleaseGate (refusal 5) · rule ReleaseGate (refusal 6)

RULE **Each requirement's status is reconciled from the tag labels of executed Allure results**

`item-04-002` **Verified, failed and untested come from the runs that carry each requirement's id**

Given Allure results: run p passed tagged item-01-001, run f failed tagged item-01-002, run s skipped tagged item-01-003, run b broken tagged item-01-004
When the requirements item-01-001 to item-01-005 are reconciled against them
Then item-01-001 is verified
And item-01-002 and item-01-004 have failed
And item-01-003 and item-01-005 are untested
When a failing run names item-01-001 only as its story, feature and epic labels

Then item-01-001 is still verified: only a tag label claims a requirement
When a passing run carries the tag label “@item-01-005”
Then item-01-005 is verified: the “@” before the id is read
When item-01-001 to item-01-003 address the need Release, item-01-004 and item-01-005 the need Trace, and verify writes the verification data
Then the traceability matrix rendered from it shows each requirement’s status under the need it addresses

Covers: rule Verify (success)

RULE **A release's evidence is retained as one bundle, generated from the record, never mixed with an earlier one**

`item-04-003` **The evidence bundle holds the verification data, the rendered matrix, the report and the results**

Given a record with a spec, a traceability matrix template and Allure results with attachments and a container
When the release manager writes the evidence bundle to an output directory apart from the results
Then the bundle holds the verification data computed from the executed results
And the traceability matrix rendered from it, with no template markers left
And the executed results with every attachment and container they reference
And the verification report, or the manifest says why it is not there
And a manifest listing exactly the files bundled
And a symbolic link among the results is not followed, and a missing attachment is listed in the manifest
And an unreadable result is named in the manifest
When the bundle is written again over the earlier one
Then its results replace the earlier bundle’s, never mixed with them
When the output directory would hold the results or sit inside them
Then the bundle is refused and the results are left untouched
When the record has no traceability matrix template
Then the bundle is refused, naming the missing template, and no bundle is written

Covers: rule EvidenceBundle (success) · rule EvidenceBundle (refusal 1) · rule EvidenceBundle (refusal 2)

RULE **RDM's gates can be reused in another repository's CI, pinned to one RDM revision**

`item-04-004` **The reusable workflow and the gates action run RDM’s gates at the revision the caller pins**

Given the composite gates action, the reusable gates workflow and the documents action
When the gates action is run with the checklist part11_document_control over a record with results
Then it runs verify, the release gate, graph validation and the evidence bundle with the verification report, in that order
And graph validation holds the documents to the checklists named
And a gate that fails fails the action

And with the release gate off and no results yet, no gate runs
And every rdm command the workflow and the actions run is one RDM's command line accepts
And the action installs RDM from its own revision, the workflow from the revision the caller pins, never from a package index
And the reusable workflow runs the caller's acceptance tests before the gates
And the documents action renders with the image of the release it is pinned to, else the latest image
And the workflow rdm adopt lays down calls the reusable workflow pinned to the installed RDM's version
And RDM's own CI calls the same workflow, pinned to the commit under test

Covers: guarantee GatesReusableInCi

6.5 risk

Feature: Risk

The accepted scenarios of this context: the risk register kept in the record, each risk scored only against the risk policy the project declares, each control a requirement of the spec, and the release held back while a risk is unscored, uncontrolled, or residually unacceptable without a named acceptance. Each is stamped with an item id that is never renumbered or reused, and verified by the acceptance tests whose Allure results carry that id as a tag label.

RULE **The register is read from kind: risk frontmatter and evaluated only against a declared policy**

`item-02-001` **Every field of a risk is read, and its levels come only from the declared risk policy**

Given dhf/documents/risk/risks.md, of kind: risk and status proposed, declares RISK-A-1, RISK-A-2 and RISK-A-3
And RISK-A-1 is a security risk with STRIDE category Spoofing, linked to RISK-A-2, hazard "Hz", situation "Si", harm "Ha", scored Serious × Possible with recorded level High, controlled by item-01-001 and item-01-002, with residual probability Unlikely, accepted by "QA lead" because "ALARP", and status approved
And RISK-A-2 is a safety risk scored Serious × Possible, controlled by item-01-001, with residual Minor × Possible and no status of its own
And RISK-A-3 is a safety risk scored Minor × Rare that nothing controls
And dhf/documents/notes.md declares a risk RISK-NOT-1 but is not of kind: risk
When the register is read with no risk policy declared
Then it holds RISK-A-1, RISK-A-2 and RISK-A-3 and not RISK-NOT-1
And every field of RISK-A-1 is read as written, its residual severity left unset, its document dhf/documents/risk/risks.md
And RISK-A-2 has the document's status, proposed
And no risk has a level or a residual level: nothing is evaluated without a policy
When dhf/documents/risk/policy.md declares a risk policy with status proposed, in which Medium is acceptable only with a justification

Then RISK-A-1 is High, residual Medium: its residual severity defaults to the initial Serious
And RISK-A-2 is High, residual Medium, from its own residual Minor × Possible
And RISK-A-3, uncontrolled, is Low and residual Low
When the policy names its own levels Green, Amber and Red and a risk scored Bad × Often has residual probability Seldom
Then that risk is Red, residual Amber
When the policy gives no acceptability, leaves a severity and probability pair without a level, calls a level “fine”, or lists a severity or probability twice
Then the policy is refused, naming documents/risk/policy.md
When a second document, documents/risk/second_policy.md, also declares a risk policy
Then the policy is refused, naming both documents

Covers: guarantee RegisterReadFromFrontmatter

RULE **The release gate blocks a register it cannot evaluate as written**

`item-02-002` **A register or policy that cannot be evaluated as written blocks release, naming the risk or the document**

Given a spec whose requirement item-01-001 has a passing run and item-01-002 none
And in the record: <register>
When the release manager runs the release gate
Then its risk findings are <findings>
And each finding about one risk names that risk

Examples: the register’s own rules

case	register	findings
sound	the risk policy is declared; five sound risks, one a security risk of STRIDE category Tampering linked to another, two accepted by “QA lead” because “ALARP”, one whose control lowers the residual severity	none
no-policy	no risk policy is declared; RISK-P-1	“the risk register has risks but no risk_policy is declared (acceptability criteria missing)” and “risk RISK-P-1 cannot be evaluated: no risk policy”
duplicate	the risk policy is declared; RISK-D-1 declared twice	“risk RISK-D-1 is declared 2 times”

case	register	findings
no-hazard	the risk policy is declared; RISK-C-1 with an empty hazard	“risk RISK-C-1 has no hazard”
no-situation	the risk policy is declared; RISK-C-2 with no situation	“risk RISK-C-2 has no situation”
no-harm	the risk policy is declared; RISK-C-3 whose harm is blank	“risk RISK-C-3 has no harm”
untyped-parts	the risk policy is declared; RISK-C-4 whose hazard, situation and harm are not text	“risk RISK-C-4 has no hazard”, “risk RISK-C-4 has no situation” and “risk RISK-C-4 has no harm”
no-category	the risk policy is declared; RISK-B-1 with no category	one, naming RISK-B-1, that “needs a category of safety or security”
bad-category	the risk policy is declared; RISK-B-2 of category ops	one, naming RISK-B-2, that “needs a category of safety or security (got ops)”
no-stride	the risk policy is declared; RISK-B-3, a security risk with no STRIDE category	one, naming RISK-B-3, that “is a security risk with no STRIDE category”
bad-stride	the risk policy is declared; RISK-B-4, a security risk of STRIDE category Hacking	one, naming RISK-B-4, with “no STRIDE category”
bad-link	the risk policy is declared; RISK-B-5 linked to RISK-NOPE-9	“risk RISK-B-5 links RISK-NOPE-9, which is not a declared risk”
unknown-severity	the risk policy is declared; RISK-S-1 of severity Awful	one, naming RISK-S-1, that “is not defined by the risk policy”
unknown-probability	the risk policy is declared; RISK-S-2 with no probability	one, naming RISK-S-2, that “is not defined by the risk policy”
mis-scored	the risk policy is declared; RISK-S-3, Serious × Possible, recording level Low	“risk RISK-S-3 records level Low; the risk policy says High”

case	register	findings
mis-scored-residual	the risk policy is declared; RISK-S-4, residual probability Rare, recording residual level Block	“risk RISK-S-4 records residual level Block; the risk policy says Low”
scalar-control	the risk policy is declared; RISK-U-5 with the single control item-01-077 and the single link RISK-NOPE-9	“risk RISK-U-5 names control item-01-077, which is not a requirement of the spec” and “risk RISK-U-5 links RISK-NOPE-9, which is not a declared risk”
undeclared-control	the risk policy is declared; RISK-U-2 controlled by item-01-001 and item-01-077	“risk RISK-U-2 names control item-01-077, which is not a requirement of the spec”
no-residual	the risk policy is declared; RISK-R-1 with controls and no residual	“risk RISK-R-1 has controls but no residual score”
bad-residual	the risk policy is declared; RISK-R-2 with residual probability Never	one, naming RISK-R-2, that “residual ‘Serious’ × ‘Never’ is not defined”
no-register	no risk document at all, and no risk policy	none: the policy is required only when there are risks
anonymous	the risk policy is declared; a risk with no id	“a risk in dhf/documents/risk/risks.md has no id”
malformed-policy	documents/risk/policy.md declares risk_policy: [1, 2]; a risk	one naming “risk_policy in documents/risk/policy.md”
policy-twice	the risk policy is declared in documents/risk/policy.md and again in documents/risk/second_policy.md; a risk	one that “risk_policy is declared twice”, naming second_policy.md
risks-a-mapping	the risk policy is declared; a document of kind: risk whose risks are a mapping of id to risk	release blocked, naming documents/risk/risks.md
risks-strings	the risk policy is declared; a document of kind: risk	release blocked, naming documents/risk/risks.md

case	register	findings
	whose risks are the string RISK-M-2	
risks-misspelled	the risk policy is declared; a document of kind: risk that lists its risks under risk:	release blocked, naming documents/risk/risks.md
no-risks	the risk policy is declared; a document of kind: risk with no risks	release blocked, naming documents/risk/risks.md
capital-kind	the risk policy is declared; a document of kind: Risk whose RISK-M-3, Critical × Likely, nothing controls	release blocked, naming RISK-M-3: its kind is read in any case
policy-only	a document of kind: risk that only declares the approved risk policy; item-01-001 and item-01-002 pass	none, and release passes: it is not a register
list-id	the risk policy is declared; a risk whose id is the list [RISK-I-1], and RISK-I-1	“a risk in dhf/documents/risk/risks.md has an id that is not text or a number: [‘RISK-I-1’]”
numeric-severities	a policy whose severities are the numbers 1 and 2; RISK-N-1 of severity 2 × Likely, residual Rare; item-01-001 and item-01-002 pass	none, and release passes

Covers: guarantee MalformedRegisterBlocks

RULE **A risk is released only with an evaluated, acceptable or accepted residual and a known status**

`item-02-003` **An unevaluated, unacceptable or unaccepted residual blocks release; a proposed rating only warns**

Given a spec whose requirement item-01-001 has a passing run and item-01-002 none
And in the record: <register>
When the release manager runs the release gate
Then its risk findings are <findings>

Examples: the residual decision and the status

case	register	findings
unverified-control	the risk policy is declared; RISK-R-6 controlled by item-01-001 and item-01-002, which has no passing run	“risk RISK-R-6: residual not evaluated — control item-01-002 has no passing test”
uncontrolled-unacceptable	the risk policy is declared; RISK-U-1, Critical × Likely, that nothing controls, accepted by “QA lead” because “ALARP”	“risk RISK-U-1 has an unacceptable residual level of Block”: no acceptance makes Block pass
uncontrolled-claimed-residual	the risk policy is declared; RISK-U-4, Critical × Likely, that nothing controls, claiming residual Negligible × Rare	“risk RISK-U-4 has an unacceptable residual level of Block”: only a control reduces a risk
uncontrolled-unaccepted	the risk policy is declared; RISK-U-3, Serious × Unlikely, that nothing controls, with no acceptance	one, naming RISK-U-3, that “has a residual level of Medium that needs an acceptance”
block-residual	the risk policy is declared; RISK-R-3, Critical × Likely, residual probability Likely, accepted	“risk RISK-R-3 has an unacceptable residual level of Block”
unaccepted-medium	the risk policy is declared; RISK-R-4 with residual probability Unlikely and no acceptance	one, naming RISK-R-4, that “has a residual level of Medium that needs an acceptance”
half-accepted-high	the risk policy is declared; RISK-R-5 with residual probability Possible, accepted by “QA” with no reason	one, naming RISK-R-5, that “has a residual level of High that needs an acceptance”
acceptance-not-text	the risk policy is declared; RISK-R-7 with residual probability Unlikely, accepted by an empty list because “ALARP”	one, naming RISK-R-7, that “has a residual level of Medium that needs an acceptance”

case	register	findings
acceptance-not-words	the risk policy is declared; RISK-R-8 with residual probability Unlikely, accepted by “QA lead” because true	one, naming RISK-R-8, that “has a residual level of Medium that needs an acceptance”
bad-status	the risk policy is declared; RISK-T-1 of status maybe	“risk RISK-T-1 has an unknown status ‘maybe’ (proposed or approved)”
proposed	the risk policy, of status proposed; RISK-W-1 of status proposed and RISK-W-2 approved; item-01-001 and item-01-002 pass	none, and release passes, warning “risk RISK-W-1 is proposed: a person has not approved its rating” and “the risk policy in documents/risk/policy.md is proposed: a person has not approved it”, and nothing about RISK-W-2
no-status	the risk policy, with no status; RISK-W-3 of empty status and RISK-W-4 with none; item-01-001 and item-01-002 pass	none, and release passes, warning that RISK-W-3, RISK-W-4 and the risk policy are proposed: no status is never approved

Covers: guarantee UncontrolledRiskBlocks

6.6 specification

Feature: Specification

The accepted scenarios of this context: how RDM reads the spec it holds a record to (its requirements and the user needs they address), how a project is scaffolded or an existing repository adopted, and how validation by a person shows at release. Each is stamped with an item id that is never renumbered or reused, and verified by the acceptance tests whose Allure results carry that id as a tag label.

RULE **The spec's requirements are its Gherkin scenarios tagged with an id, read whole and never dropped quietly**

`item-01-001` **Every tagged scenario is a requirement, wherever the spec’s layout keeps it**

Given a Löyly plan whose slices are 03-gate, 04-stamps, 05-later with status planned and 06-someday with status backlog

And 03-gate's spec.feature holds, under the Rule "Booting", a scenario "boots fresh" tagged @allium-rule-success.BootFresh @allium-guarantee.Fresh @item-03-001, a scenario " refuses a busy seam" tagged @item-03-009 @item-03-002, and an untagged scenario "has no stamp"
And 03-gate has a drafts.feature, and an entry "notes" that is no slice holds a spec.feature
When the spec is read
Then its requirements are item-03-001, item-03-002 and item-04-001, in plan order, in the contexts gate and stamps
And 05-later and 06-someday are skipped and recorded as planned and backlog, and the drafts and "notes" are never read
And item-03-001 keeps its Rule "Booting", its title and its source line ".pi/plan/03-gate/spec.feature:6"
And the second scenario's id is item-03-002, the last of its item tags, and its title is trimmed to "refuses a busy seam"
And item-04-001 has no Rule: a Rule never carries over from another file
And item-03-001's Allium obligations are rule-success BootFresh and guarantee Fresh
And "has no stamp" is listed as unstamped at ".pi/plan/03-gate/spec.feature:14", and no id is stamped twice
And reading the spec imports no planning tool
And Löyly's own spec, when present, reads as its checklist golden: 201 stamped items, per context
Given instead a repository with no spec configuration and feature files under features/
And features/alarms/sound.feature holds, under the Rule "Thresholds", a scenario "a high heart rate alarms" tagged @smoke @REQ-1, and an untagged scenario "not yet tagged"
And features/top.feature holds a scenario "at the root" tagged @REQ-2
When the spec is read
Then features/ is found, and REQ-1 and REQ-2 are its requirements: any id-shaped tag is an id
And REQ-1 is in the context alarms, under the Feature "Alarms sound" and the Rule "Thresholds", sourced at features/alarms/sound.feature:7, keeping the tag @smoke
And REQ-2, at the root, has no context
And "not yet tagged" is listed as unstamped at features/alarms/sound.feature:10
When the configuration names the globs "../features/**/*.feature", excludes "top.feature" and sets the id pattern "REQ-1"
Then the only requirement is REQ-1

Covers: guarantee SpecReadAsTaggedGherkin

RULE **A project starts from one command, and an existing repository is adopted without disturbing it**

`item-01-002` **rdm init scaffolds a project that builds offline, and refuses a directory that exists**

Given the directory "regulatory" does not exist
When the regulatory author runs rdm init into "regulatory"
Then it names "regulatory", the next steps and rdm gap among them

And it lays down the Makefile, the render configuration config.yml and the documents directory
And the document templates, the software design specification, the design review and the traceability matrix among them
And no document fetches an image from the network, the Makefile rebuilds Word output when reference.docx changes, and the Dockerfile installs RDM from its git repository rather than a copied wheel
And a spec skeleton written the Löyly way: .pi/plan/01-core linking .pi/spec/core, with spec.feature, drafts.feature, allium/core.allium, event-model/area.edn and .pi/spec/event-storm.edn
And RDM reads it as one user need, “core/desk”, linked to the surface Desk, with no requirement yet
When rdm init is run into “regulatory” again
Then it is refused, saying the directory exists

Covers: rule Init (success) · rule Init (refusal 1)

`item-01-003` **rdm adopt lays down the DHF skeleton and the CI gates, skipping what exists**

Given a repository with src/app.py and its own dhf/README.md
When the regulatory author runs rdm adopt in it
Then the V&V plan and the traceability matrix template are laid down under dhf/documents
And the CI workflow calls RDM’s reusable gates workflow pinned to the RDM version that adopted it
And dhf/README.md and src/app.py are left as they were, and dhf/README.md is reported as skipped
And a .gitignore of what RDM generates (allure-results/, dhf/data/, .rdm/, __pycache__/) and dhf/config.yml are laid down
And the repository, having no spec, gets a spec skeleton written the Löyly way (.pi/plan/01-core linking .pi/spec/core), which RDM reads as the user need “core/desk”
When rdm adopt is run again
Then nothing is laid down: everything exists
Given another repository whose .gitignore holds “node_modules/” and whose dhf/config.yml is a symbolic link to a file outside it that does not exist
When rdm adopt is run in it
Then its .gitignore is kept as it was, and the lines to add to it are named
And nothing is written through the symbolic link
Given a third repository that already keeps its spec as Gherkin under spec/
When rdm adopt is run in it
Then no spec skeleton is laid down beside it

Covers: rule Adopt (success)

RULE **Validation is made by a person, and a user need without it shows at release**

`item-01-004` **The release gate warns of each user need without an approved validation record**

Given a verified record whose spec has the user needs Alarm and Log, and no validation records
When the release gate runs
Then it warns that Alarm and Log have no approved validation record, and release passes
When dhf/validation holds a record for Alarm, disposition approved, reviewer “maintainer (summative)”, and one for Log, disposition pending
Then the release gate warns of Log only, and release passes
And the records are read keyed by user need, with their disposition and reviewer, and Log is the one need unvalidated
When Alarm’s approved record names no reviewer, an empty one, or a reviewer that is not text
Then Alarm and Log are both unvalidated: a person makes a validation

Covers: guarantee UnvalidatedNeedsNamedAtRelease

RULE **The user needs come from the source the spec's configuration names**

`item-01-005` **A need is a Feature by default, or an event-model screen with its Allium surfaces and storm events**

Given plain Gherkin under spec/alarms: the Feature “Alarms sound” described “A nurse is alerted to a dangerous vital.” holding REQ-1, and the Feature “Quiet hours” holding REQ-2
When the spec is read
Then its needs are “Alarms sound” and “Quiet hours”, in the context alarms, “Alarms sound” with its description as its text
And REQ-1 addresses “Alarms sound”, and “Quiet hours” is addressed by REQ-2
Given instead a Löyly plan with an event storm, a context desk whose screen operator-desk (“The operator’s view: derive a checklist.”) links the surface Desk, facing an Operator, providing Derive, guaranteeing NeverWritten, with the command derive that produces the storm event “Checklist derived” an Operator causes
And desk’s Allium also has a surface Seam that no screen links, and a context report with no screens whose surface Report (“An actor reads the report.”) faces a Reader
And item-01-001 is tagged @allium-rule-success.Derive, item-01-002 @allium-rule-failure.Unrelated, item-02-001 @allium-guarantee.Readable
When the spec is read
Then desk/operator-desk is a need with that text, the surface Desk, the actor Operator, the rule Derive, the guarantee NeverWritten, the command desk/derive and the storm event “Checklist derived (Operator)”
And Seam is no need: a context with screens takes its needs from them
And Report is a need, its doc comment its text, its actor Reader
And item-01-001 addresses desk/operator-desk directly, item-01-002 through its context, and item-02-001 addresses Report
And Löyly’s own spec, when present, has six needs, one screen per context, each directly addressed

Covers: guarantee SpecNeedsReadFromTheirSource

6.7 test_evidence

Feature: Test evidence

The accepted scenarios of this context: how executed Allure results are read as test runs, and what an acceptance run records about the version it tested and who ran it. Each is stamped with an item id that is never renumbered or reused, and verified by the acceptance tests whose Allure results carry that id as a tag label.

RULE **A result RDM cannot read is never counted as a pass**

`item-03-001` **An unreadable result blocks release and is named, and a failed step fails its run**

Given a spec whose only requirement is item-01-001, addressing the need Release
And a passing Allure result “a” tagged item-01-001
Then the release gate passes
When a result “b” is added that cannot be read, holding any of:

content
JSON cut short after ‘“value”: “item-01-001”’
a JSON list, not an object
bytes that are not UTF-8

Then release is blocked: the result is unreadable, and it could hold a failed run
When result “b” is a failed run tagged item-01-001 written with a byte-order mark
Then it is read, and release is blocked: item-01-001 failed
When a result “b” tagged item-01-001, beside the passing result “a”, has any of:

case	what is wrong
capital-status	the status “Failed”, which Allure does not write
no-status	no status
null-status	a null status
labels-text	labels given as the text “tag=item-01-001”
tag-list	a tag label whose value is a list
link	it is a symbolic link to a result elsewhere
deep	JSON nested 100000 levels deep

Then release is blocked by that unreadable result alone
When failing runs beside the passing result “a” are tagged “Item-01-001”, “item_01_001”, “item–01-001” and “item-01-777”
Then release passes, and the gate warns “Allure result tag <tag> matches no requirement” for each of the four
When a run marked passed has an inner step that failed, and another an inner step that broke

Then both runs have failed, and so have the requirements they claim
When a result “x-result.json” cannot be read and verify is run
Then verify exits non-zero and the verification data lists x-result.json as unreadable

Covers: guarantee UnreadableResultNeverPasses

RULE **A run that claims a requirement carries the commit it tested**

`item-03-002` **The pytest plugin labels each run of a claiming test with the commit under test**

Given a committed repository whose spec holds item-01-001 and item-01-002, with its DHF in sw/dhf
And tests test_alarm tagged item-01-001, test_log tagged item-01-002, test_undeclared tagged item-01-099 and test_untagged with no tag
When the tests run with rdm.pytest_plugin on a clean working tree
Then the results of test_alarm and test_log carry the commit label of the repository’s HEAD
And test_alarm carries no worktree label
And test_undeclared and test_untagged carry no commit label
When the test file has an edit not yet committed and the tests run again
Then test_alarm carries the worktree label “dirty” and still the commit label of HEAD

Covers: guarantee RunLabelledWithTheCommitUnderTest

RULE **The results of an acceptance run record who ran it and where**

`item-03-003` **executor.json and environment.properties name the executor and the environment**

Given a committed repository whose two tests claim item-01-001
When the tests run with rdm.pytest_plugin on GitHub Actions, run 7001 attempt 2, run number 42 of the workflow “Design controls” in acme/device, by octocat on a pull_request, on a Linux X64 runner
Then executor.json names “GitHub Actions” of type github, build order 42, the build name “Design controls #42” and the URL https://github.com/acme/device/actions/runs/7001/attempts/2
And environment.properties records the operating system, the Python, pytest, allure-pytest and RDM versions
And the commit under test with the worktree “clean”
And ci.actor octocat, ci.workflow “Design controls”, ci.event pull_request and ci.runner “Linux X64”, and no user
When the tests run locally with an uncommitted file in the repository
Then executor.json names a local run with the build name “<user>@<host>”
And environment.properties records the user and no ci. entry
And the commit under test with the worktree “dirty”
And executor.json and environment.properties sit beside the two tests’ results
When the tests run from a copy of the repository outside git
Then the commit and the worktree are both recorded as “unknown”

Covers: guarantee RunRecordsWhoRanItAndWhere

7 Assurance activities

Proportionate to that risk ([[CSA:assurance]]):

- **Scripted, automated testing.** Each of RDM's 44 requirements (stamped Given/When/Then scenarios in `.pi/spec/`) is verified by an acceptance test that claims it; RDM's own release gate blocks while any requirement lacks a passing run, any user need is unaddressed, or any risk control is unverified.
- **Risk-based focus.** The requirements that control one of the risks (the release gate, the read-only agent server, the served graph) count only once verified and the risk's residual is evaluated again.
- **Scenario testing on a real product.** The worked example runs RDM's gates over Löyly's pi harness (201 requirements, six clojure.test suites, a spec in another repository) on every change to RDM, beside RDM's own record.
- **Ongoing monitoring.** Every change runs every gate in CI, on RDM's own record (dogfooding) and on the example; a gate that stops passing stops the change.

The assurance also relies on ([[CSA:assurance-considerations]]): the dependencies pinned in `uv.lock`, the RDM container image built from them, and the executed results RDM keeps as evidence rather than screenshots.

8 Record

The record of each validation ([[CSA:record]], [[13485:4.1.6-records]]) is the evidence bundle the design-controls workflow writes from each run: the verification report (the commit validated, who and what ran the tests and where, each requirement with its runs, the issues found, and whether the evidence is release-grade), the traceability matrix, the verification data and the results with a manifest and one SHA-256 over them. The conclusion is the release gate's verdict; the approval, the reviewed pull request.

9 The record system and Part 11

The validation record is required as evidence of a validated state and is kept electronically, so it is a Part 11 electronic record and the system that keeps it is held to Part 11 ([[CSA:part11]]): git as the document control system, GitHub as its service provider and RDM's gates, mapped control by control in DC-001. That system is validated by this same procedure: RDM's gates are what the assurance above tests.

10 Change

RDM is validated before a release is first used and again after every change ([[13485:4.1.6-before-use]], [[CSA:change]]): every pull request runs every gate before it can merge; a release is a tag, and a team adopting RDM pins its CI to one release (the reusable workflow and the image, item-04-004), so a new RDM release reaches a team only when it changes the pin, and the team assesses that change for its own use ([[13485:7.5.6-software]] where RDM is used in production).